EDA

Vance Russell[11] 3point.xyz

A comprehensive guide to understanding your data through exploratory data analysis (EDA) that includes systematic exploration, visualization, and statistical analysis.

---

1. What is EDA?

EDA is the critical first step in any data analysis project. It involves:

- **Understanding structure** - How is the data organized?
- **Identifying patterns** - What trends or relationships exist?
- **Detecting anomalies** - Are there outliers or errors?
- **Assessing data quality** - Missing values, duplicates, inconsistencies?
- **Generating hypotheses** - What insights might be worth investigating?

EDA is iterative and visual—your goal is to understand the data deeply before modeling or making conclusions.

---

2. Python Packages for EDA

2.a. Core Tools

Package	Purpose	R Equivalent
pandas	Data loading, manipulation, basic profiling	base R, dplyr
numpy	Numerical computations, statistics	base R
matplotlib, seaborn	Visualization	ggplot2, base graphics
ydata-profiling	Automated profiling reports	**DataExplorer** ⭐
missingno	Missing value visualization	vis_miss()

Note: `ydata-profiling` (formerly pandas-profiling) is the closest Python equivalent to R's DataExplorer package. It generates comprehensive HTML reports with minimal code.

Published Apr 16, 2026

```python
# Import essential libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno
```

```
# Optional: Uncomment if you have ydata-profiling installed
# from ydata_profiling import ProfileReport

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
pd.set_option('display.width', None)

# Visualization settings
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

print("✓ Libraries imported successfully!")
```

3. Section 1: Creating Sample Data

We'll create a realistic geospatial/environmental dataset for this tutorial:

```
# Create a realistic environmental monitoring dataset
np.random.seed(42)
n_samples = 200

data = {
    'Site_ID': [f'SITE_{i:03d}' for i in range(1, n_samples + 1)],
    'Year': np.random.choice([2020, 2021, 2022, 2023], n_samples),
    'Latitude': np.random.uniform(35, 40, n_samples),
    'Longitude': np.random.uniform(-120, -115, n_samples),
    'Elevation_m': np.random.normal(2000, 500, n_samples),
    'Temperature_C': np.random.normal(15, 5, n_samples),
    'Precipitation_mm': np.random.exponential(50, n_samples),
    'Forest_Cover_%': np.random.normal(60, 20, n_samples),
    'Population_Density': np.random.exponential(100, n_samples),
    'Protected_Status': np.random.choice(['Protected', 'Unprotected',
'Buffer'], n_samples),
    'Land_Use': np.random.choice(['Forest', 'Agricultural', 'Urban',
'Shrubland'], n_samples)
}

df = pd.DataFrame(data)

# Introduce some missing values
df.loc[np.random.choice(df.index, 10), 'Temperature_C'] = np.nan
df.loc[np.random.choice(df.index, 8), 'Precipitation_mm'] = np.nan
df.loc[np.random.choice(df.index, 5), 'Forest_Cover_%'] = np.nan

# Add some duplicates
df = pd.concat([df, df.iloc[:5]], ignore_index=True)

print(f"Dataset created: {df.shape[0]} rows × {df.shape[1]} columns")
print(df.head())
```

4. Section 2: Basic Data Structure and Inspection

4.a. *First Look at the Data*

```
# Basic shape and structure
print("=" * 60)
print("DATASET STRUCTURE")
print("=" * 60)
print(f"Shape: {df.shape} ({df.shape[0]} rows, {df.shape[1]} columns)")
print(f"\nColumn Names and Types:")
print(df.dtypes)

# More detailed info
print("\n" + "=" * 60)
print("DETAILED INFO")
print("=" * 60)
df.info()

# First and last rows
print("\n" + "=" * 60)
print("FIRST FEW ROWS")
print("=" * 60)
print(df.head())

print("\nLAST FEW ROWS")
print(df.tail())
```

5. Section 3: Missing Values Analysis

Missing data can bias results or reduce sample size. Always investigate!

5.a. *Summary Statistics*

```
# Missing value analysis
print("=" * 60)
print("MISSING VALUES")
print("=" * 60)

# Count and percentage
missing = pd.DataFrame({
    'Count': df.isnull().sum(),
    'Percentage': (df.isnull().sum() / len(df) * 100).round(2)
})
missing = missing[missing['Count'] > 0].sort_values('Count',
ascending=False)

if len(missing) > 0:
    print(missing)
else:
    print("No missing values found (but we introduced some, so they
should appear)")

# Total missing
total_cells = df.shape[0] * df.shape[1]
total_missing = df.isnull().sum().sum()
print(f"\nTotal missing cells: {total_missing} / {total_cells}
```

```
({total_missing/total_cells*100:.2f}%)")

# Duplicates
print("\n" + "=" * 60)
print("DUPLICATES")
print("=" * 60)
duplicates = df.duplicated().sum()
print(f"Duplicate rows: {duplicates}")

if duplicates > 0:
    print(f"\nDuplicate rows:")
print(df[df.duplicated(keep=False)].sort_values(by=list(df.columns)))
```

5.b. *Visualizing Missing Values with Missingno*

```
# Visualize missing values
fig, axes = plt.subplots(2, 2, figsize=(14, 8))

# Matrix visualization - shows missing pattern
plt.subplot(2, 2, 1)
msno.matrix(df, ax=plt.gca(), sparkline=False)
plt.title('Missing Value Matrix', fontweight='bold')

# Bar chart - shows count by column
plt.subplot(2, 2, 2)
msno.bar(df, ax=plt.gca())
plt.title('Missing Value Counts', fontweight='bold')

# Heatmap - shows correlation of missingness
plt.subplot(2, 2, 3)
msno.heatmap(df, ax=plt.gca())
plt.title('Missing Value Correlation', fontweight='bold')

plt.tight_layout()
plt.show()

print("Interpretation:")
print("- Matrix: White lines show missing data locations")
print("- Bar: Heights show how many values are present in each column")
print("- Heatmap: Shows which missing values tend to occur together")
```

6. Section 4: Descriptive Statistics

6.a. *Numerical Columns*

```
# Statistical summary of numerical columns
print("=" * 60)
print("DESCRIPTIVE STATISTICS")
print("=" * 60)
print(df.describe().round(2))

# Get specific statistics
print("\n" + "=" * 60)
print("SKEWNESS AND KURTOSIS")
```

```
print("=" * 60)
numeric_cols = df.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    skew = df[col].skew()
    kurt = df[col].kurtosis()
    print(f"{col:25} - Skewness: {skew:7.2f}  |  Kurtosis: {kurt:7.2f}")

# Categorical summary
print("\n" + "=" * 60)
print("CATEGORICAL VARIABLES")
print("=" * 60)
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    print(f"\n{col}:")
    print(df[col].value_counts())
    print(f"Unique values: {df[col].nunique()}")
```

7. Section 5: Distribution Analysis

7.a. Visualizing Distributions

```
# Create histograms and box plots for numeric columns
numeric_cols = df.select_dtypes(include=[np.number]).columns

fig, axes = plt.subplots(4, 2, figsize=(14, 12))
axes = axes.flatten()

for idx, col in enumerate(numeric_cols[:8]):
    # Histogram with KDE
    axes[idx].hist(df[col].dropna(), bins=20, edgecolor='black',
alpha=0.7, color='steelblue')
    axes[idx].set_title(f'{col}\n(n={df[col].notna().sum()},
missing={df[col].isna().sum()})',
                        fontweight='bold', fontsize=10)
    axes[idx].set_ylabel('Frequency')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Box plots for outlier detection
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.flatten()

for idx, col in enumerate(numeric_cols[:6]):
    sns.boxplot(y=df[col], ax=axes[idx], color='lightblue')
    axes[idx].set_title(f'{col}', fontweight='bold')
    axes[idx].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("Interpretation:")
```

```
print("- Histograms show the distribution shape")
print("- Box plots show median, quartiles, and potential outliers")
print("- Points beyond whiskers may be outliers worth investigating")
```

8. Section 6: Correlation and Relationships

8.a. Correlation Matrix

```
# Calculate correlation matrix
corr_matrix = df.corr(numeric_only=True)

# Heatmap visualization
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8}, ax=ax)
ax.set_title('Correlation Matrix: Numerical Variables', fontsize=13,
fontweight='bold', pad=15)
plt.tight_layout()
plt.show()

# Find strongest correlations (excluding diagonal)
print("=" * 60)
print("STRONGEST CORRELATIONS")
print("=" * 60)

# Get correlations above threshold (excluding 1.0)
corr_list = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        corr_list.append({
            'Variable1': corr_matrix.columns[i],
            'Variable2': corr_matrix.columns[j],
            'Correlation': corr_matrix.iloc[i, j]
        })

corr_df = pd.DataFrame(corr_list).sort_values('Correlation', key=abs,
ascending=False)
print(corr_df.head(10).to_string(index=False))
```

9. Section 7: Automated Profiling with ydata-profiling

The `ydata-profiling` package (formerly pandas-profiling) is the Python equivalent of R's DataExplorer. It generates comprehensive HTML reports with one line of code.

9.a. Installation

```
pip install ydata-profiling
```

9.b. Quick Example

10. Section 7: Automated Profiling with ydata-profiling

The `ydata-profiling` package (formerly pandas-profiling) is the Python equivalent of R's DataExplorer. It generates comprehensive HTML reports with one line of code.

10.a. *Installation*

```
pip install ydata-profiling
```

10.b. *Quick Example*

```
# Example: Automated profiling with ydata-profiling
# Uncomment below to run (requires: pip install ydata-profiling)

"""
from ydata_profiling import ProfileReport

# Generate profile report
profile = ProfileReport(df, title="Environmental Data Profile",
minimal=False)

# Save to HTML file
profile.to_file("eda_report.html")

# Or display in Jupyter
# profile.to_notebook_iframe()
"""

# Manual equivalent when ydata-profiling is not available
print("=" * 60)
print("EDA SUMMARY (Manual Alternative to ydata-profiling)")
print("=" * 60)

summary = {
    'Total Rows': len(df),
    'Total Columns': len(df.columns),
    'Numeric Columns':
len(df.select_dtypes(include=[np.number]).columns),
    'Categorical Columns':
len(df.select_dtypes(include=['object']).columns),
    'Total Missing': df.isnull().sum().sum(),
    'Duplicate Rows': df.duplicated().sum(),
    'Memory Usage (MB)': df.memory_usage(deep=True).sum() / 1024**2
}

for key, value in summary.items():
    if isinstance(value, float):
        print(f"{key:.<35} {value:.2f}")
    else:
        print(f"{key:.<35} {value}")
```

11. Section 8: Categorical Data Analysis

11.a. *Distribution of Categories*

```
# Categorical data visualization
categorical_cols = df.select_dtypes(include=['object']).columns

fig, axes = plt.subplots(1, len(categorical_cols), figsize=(14, 4))
if len(categorical_cols) == 1:
```

```
    axes = [axes]

for idx, col in enumerate(categorical_cols):
    value_counts = df[col].value_counts()
    axes[idx].barh(value_counts.index, value_counts.values,
color='steelblue', edgecolor='black')
    axes[idx].set_title(f'{col}\n(n={len(value_counts)} categories)',
fontweight='bold')
    axes[idx].set_xlabel('Count')
    axes[idx].grid(True, alpha=0.3, axis='x')

plt.tight_layout()
plt.show()

# Cross-tabulation analysis
print("=" * 60)
print("CROSSTAB: Protected_Status × Land_Use")
print("=" * 60)
if 'Protected_Status' in df.columns and 'Land_Use' in df.columns:
    crosstab = pd.crosstab(df['Protected_Status'], df['Land_Use'],
margins=True)
    print(crosstab)
```

12. Section 9: EDA Checklist and Best Practices

12.a. The Complete EDA Workflow

```
1. ✓ Load Data
   - df.shape, df.head(), df.info()
   - Check for obvious issues

2. ✓ Data Quality
   - Missing values (count, pattern, mechanism)
   - Duplicates
   - Data types and ranges

3. ✓ Univariate Analysis
   - Distributions (histograms, box plots)
   - Descriptive statistics (mean, median, std, skew)
   - Outliers

4. ✓ Bivariate Analysis
   - Relationships between variables
   - Correlation matrix
   - Scatter plots, grouped box plots

5. ✓ Multivariate Analysis
   - Patterns across multiple dimensions
   - Grouping and segmentation
   - Dimension reduction (PCA if applicable)

6. ✓ Categorical Analysis
   - Frequency distributions
```

```
   - Cross-tabulations
   - Associations

7. ✓ Generate Insights
   - Hypotheses for further investigation
   - Data quality issues to address
   - Next steps for modeling
```

---

13. Section 10: Reusable EDA Function

Create a function you can use on any dataset:

```python
def quick_eda(df, name="Dataset"):
    """
    Perform quick EDA on a DataFrame

    Parameters:
    -----------
    df : pandas.DataFrame
        The dataset to analyze
    name : str
        Name of the dataset for reporting
    """
    print(f"\n{'='*70}")
    print(f"EXPLORATORY DATA ANALYSIS: {name}")
    print(f"{'='*70}\n")

    # Shape and types
    print(f"Shape: {df.shape[0]} rows × {df.shape[1]} columns")
    print(f"Memory: {df.memory_usage(deep=True).sum() / 1024**2:.2f}
MB\n")

    # Missing values
    missing_pct = (df.isnull().sum() / len(df) * 100)
    if missing_pct.sum() > 0:
        print("Missing Values:")
        print(missing_pct[missing_pct > 0].sort_values(ascending=False))
        print()

    # Duplicates
    print(f"Duplicates: {df.duplicated().sum()}")

    # Data types
    print("\nData Types:")
    print(df.dtypes.value_counts())

    # Numerical summary
    print("\n" + "-"*70)
    print("NUMERICAL VARIABLES")
    print("-"*70)
    print(df.describe().round(2))
```

```python
    # Categorical summary
    cat_cols = df.select_dtypes(include=['object']).columns
    if len(cat_cols) > 0:
        print("\n" + "-"*70)
        print("CATEGORICAL VARIABLES")
        print("-"*70)
        for col in cat_cols:
            print(f"\n{col}: {df[col].nunique()} unique values")
            print(df[col].value_counts().head(3))

    print(f"\n{'='*70}\n")

# Test the function
quick_eda(df, name="Environmental Monitoring Data")
```

14. Summary: Key EDA Takeaways

14.a. Tools and Packages

Tool	Best For	Link
pandas	Core data manipulation & basic stats	https://pandas.pydata.org/
numpy	Numerical operations	https://numpy.org/
matplotlib/seaborn	Visualizations	https://seaborn.pydata.org/
missingno	Missing value patterns	https://github.com/ResidentMario/missingno
ydata-profiling	Automated reports (R's DataExplorer equivalent)	https://github.com/ydataai/ydata-profiling

14.b. Critical Questions to Answer in EDA

1. **Data Quality:** How much missing data? Duplicates? Errors?
2. **Distributions:** Are variables normally distributed? Skewed?
3. **Relationships:** Which variables correlate? Are there patterns?
4. **Outliers:** Are there unusual values? Are they real or errors?
5. **Categories:** How many unique values? Are categories balanced?
6. **Trends:** Are there temporal or spatial patterns?
7. **Anomalies:** What stands out? What needs investigation?

14.c. Best Practices

✓ **Always visualize** - Distributions, correlations, relationships
✓ **Document findings** - Notes, hypotheses, data quality issues
✓ **Investigate anomalies** - Don't ignore outliers or patterns
✓ **Check for biases** - Are certain groups over/under-represented?
✓ **Be iterative** - EDA is not linear; drill deeper as questions arise
✓ **Use automation** - ydata-profiling for initial overview
✓ **Clean systematically** - Document all changes made to data

14.d. Next Steps After EDA

1. **Feature Engineering** - Create new variables based on insights
2. **Data Cleaning** - Address missing values, outliers, errors
3. **Modeling** - Use EDA insights to inform model selection
4. **Validation** - Compare model results with EDA findings
5. **Communication** - Share key discoveries with stakeholders

---

Remember: EDA is the most important step in any data analysis project. Time spent here pays dividends later!

Happy exploring! 🔍📊